

WEB-BASED VEHICLE AUCTION PLATFORM

Full-Stack Developer Implementation Guide

A complete step-by-step manual for building the ThaproJapan auction platform from scratch

Stack: Node.js · Express · PostgreSQL · React 19 · Vite · Tailwind CSS v4

Table of Contents

1. Project Overview	5
1.1 What the System Does	5
1.2 User Roles	5
1.3 Technology Stack	5
2. End-to-End Sale Workflow	6
3. Development Environment Setup	7
3.1 Prerequisites — Install in this Order	7
3.2 Create the Project Structure	7
3.3 Backend Dependencies	7
3.4 Frontend Dependencies	7
3.5 Environment Variables	7
3.6 Add .gitignore	8
4. Database Design & Migrations	9
4.1 Create the Database	9
4.2 Migration Strategy	9
4.3 Migration 001 — Initial Schema	9
4.4 Migrations 002–008 — Schema Evolution	10
4.5 Apply All Migrations	10
4.6 Final Schema Reference	10
4.7 Seed the Admin User	11
5. Backend — Layered Architecture	12
5.1 Configuration & Database Pool	12
5.2 Express Application Entry	12
5.3 Authentication Middleware	12
5.4 Validation Middleware	13
5.5 Repositories	13
5.6 Key Repository Patterns	13
User Repository	13
Auction Repository	13
Vehicle Repository	13
5.7 Services	13
Auth Service	14
Auction Service — On-Read Status Correction	14
Auction Service — Reserve Price Privacy	14
Bid Service	14

5.8 Error Convention	14
5.9 The Scheduler	15
5.10 API Route Map	15
6. Frontend Architecture.....	17
6.1 Vite Configuration	17
6.2 Tailwind CSS v4 Setup	17
6.3 Application Entry — main.jsx	17
6.4 Auth Context — The Most Important Frontend File	18
6.5 Routing	18
6.6 API Layer.....	18
6.7 Cloudinary Image Upload	19
6.8 Key Components	19
ProtectedRoute.jsx.....	19
CountdownTimer.jsx	19
StatusBadge.jsx.....	19
Navbar.jsx.....	19
6.9 Page Descriptions	19
LoginPage.jsx	19
AuctionDetailPage.jsx — Most Complex Page.....	20
VehicleListPage.jsx.....	20
AdminPage.jsx.....	20
App.jsx — Won Auctions Banner	20
7. Cloudinary Setup	21
7.1 Create a Free Account.....	21
7.2 How Upload Works in the UI	21
8. Recommended Build Order.....	22
9. Common Mistakes and How to Avoid Them.....	24
9.1 Backend Mistakes	24
Forgetting ES Module Syntax.....	24
Business Logic in the Wrong Layer	24
Not Parameterizing SQL	24
Route Order in auctions.js.....	24
Not Setting type: module in package.json	24
9.2 Frontend Mistakes	24
Not Cleaning Up Intervals	24
Polling Without Cleanup.....	25

JWT Decoding vs. Fresh Data	25
Forgetting to Dispatch Custom Events	25
9.3 Database Mistakes	25
Editing Applied Migrations	25
Storing Images in the Database	25
NULL Chassis Numbers and UNIQUE Constraint	25
10. Testing Your Implementation	26
10.1 Backend API Testing with curl	26
Auth	26
Verify a User (Admin)	26
Create Vehicle and Auction (Admin)	26
Place a Bid (Verified Buyer)	26
10.2 Frontend Smoke Test Checklist	26
11. Deployment Guide	28
11.1 Server Prerequisites	28
11.2 Database Setup on Server	28
11.3 Backend Deployment	28
11.4 Frontend Build	28
11.5 Nginx Configuration	29
11.6 Production Checklist	29
12. Quick Reference	30
12.1 File Naming & Location Reference	30
12.2 Error Response Format	30
12.3 JWT Payload Structure	31
12.4 Key Design Decisions Summary	31

1. Project Overview

This guide walks you through building the ThaproJapan vehicle auction platform entirely by hand — from database schema to deployed frontend — with no reliance on AI code generation. Every command, file, and design decision is documented so you understand what you are building and why.

1.1 What the System Does

ThaproJapan is a Japanese vehicle exporter that acquires cars at auction in Japan and resells them internationally. Currently buyers submit bids via WhatsApp, which is untrackable and unscalable. This platform replaces that workflow with:

- A public vehicle catalogue with search and filtering
- A time-controlled auction system with automatic status transitions
- Verified-buyer-only bidding with minimum increment enforcement
- An admin dashboard for vehicle management, user verification, and winner selection
- A winner notification system visible on the buyer's profile

1.2 User Roles

Role	How Created	What They Can Do
Guest (public)	None — unauthenticated	Browse vehicles and auctions, view bid history
Buyer	Self-registers at /register	All of the above + place bids once admin-verified
Admin	Seeded via script only	Full CRUD on vehicles/auctions, verify users, close auctions, assign winners

1.3 Technology Stack

Layer	Technology	Why
Frontend	React 19 + Vite + Tailwind CSS v4	Component SPA model suits dynamic auction UI; Vite gives fast HMR
Backend	Node.js 20 + Express 4	Non-blocking I/O handles concurrent bids; same language as frontend
Database	PostgreSQL 16	ACID transactions critical for bid integrity; supports array columns for images
Auth	JWT (jsonwebtoken)	Stateless — no session store needed; role claims carried in token
Images	Cloudinary (free tier)	Unsigned upload from browser; returns CDN URL stored in DB
Scheduler	node-cron	Transitions auction statuses (draft→active→ended) every minute

2. End-to-End Sale Workflow

Before writing a single line of code, understand this complete workflow. Every feature you build maps to one of these steps.

1	Admin creates a Vehicle record (status: listed) with images, specs, starting price
2	Admin creates an Auction linked to that vehicle — sets start/end times, reserve price, min increment
3	Cron job runs every minute: if starts_at has passed and status is draft → set active
4	Buyer self-registers → admin verifies account → buyer can now place bids
5	Buyer places bids on active auction; each bid must exceed current highest + min_increment
6	Cron job: if ends_at has passed and status is active → set ended
6b	OR admin clicks Close Auction manually — auto-assigns highest bid as winner
7	Admin reviews bid ladder and clicks Select Winner (or the close auto-assigned one)
8	Winner page becomes public: shows winner name and winning bid amount
9	Buyer sees You Won! banner on next login, plus the auction in their Profile

3. Development Environment Setup

Complete these steps before writing any application code. Skipping any step will cause hard-to-debug errors later.

3.1 Prerequisites — Install in this Order

1. Node.js 20 LTS — download from nodejs.org. Verify with: `node --version` and `npm --version`
2. PostgreSQL 16 — download from postgresql.org. During installation, set a password for the postgres superuser — write it down. You will need it throughout this guide.
3. Git — git-scm.com. Initialize your repo: `git init vehicle-auction` && `cd vehicle-auction`
4. VS Code (recommended) — install extensions: ESLint, Prettier, PostgreSQL (by Chris Kolkman)

3.2 Create the Project Structure

Run these commands from your project root:

```
mkdir -p
Backend/src/{config,db,middleware,repositories,services,controllers,routes}
mkdir -p Backend/migrations Backend/scripts
mkdir -p Frontend/src/{api,components,context,pages/admin,router}
cd Backend && npm init -y
cd ../Frontend && npm create vite@latest . -- --template react
```

3.3 Backend Dependencies

```
cd Backend
npm install express pg bcrypt jsonwebtoken joi morgan node-cron dotenv
npm install -D nodemon
```

Add to Backend/package.json — change type to module and add scripts:

```
"type": "module",
"scripts": {
  "start": "node src/server.js",
  "dev": "nodemon src/server.js"
}
```

3.4 Frontend Dependencies

```
cd Frontend
npm install react-router-dom axios
npm install -D tailwindcss @tailwindcss/vite @tailwindcss/forms
```

3.5 Environment Variables

Create Backend/.env (never commit this file):

```
PORT=3000
DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/vehicle_auction
JWT_SECRET=replace_this_with_a_long_random_string_minimum_32_chars
JWT_EXPIRES_IN=1h
BCRYPT_ROUNDS=10
NODE_ENV=development
```

Create Frontend/.env (these are public — they appear in the compiled JS):

```
VITE_CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name  
VITE_CLOUDINARY_UPLOAD_PRESET=vehicle_img
```

💡 To generate a strong `JWT_SECRET`: run `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"` in your terminal.

3.6 Add .gitignore

```
# Root .gitignore  
node_modules/  
.env  
dist/  
.DS_Store
```


4. Database Design & Migrations

The database is the foundation of the system. Build it before any backend code. We use plain SQL migration files — run them in order. Never edit a migration file after it has been applied to any real database.

4.1 Create the Database

```
psql -U postgres
CREATE DATABASE vehicle_auction;
\q
```

4.2 Migration Strategy

Each migration file is a numbered SQL script. If you need to change the schema, you add a NEW migration file — you never edit previous ones. This preserves a complete history of how the schema evolved.

4.3 Migration 001 — Initial Schema

File: Backend/migrations/001_initial_schema.sql

This creates the four core tables. Note the CHECK constraints on status columns — these enforce valid values at the database level, not just in application code.

```
CREATE TABLE IF NOT EXISTS users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  name VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL CHECK (role IN ('admin', 'buyer')),
  is_verified BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS vehicles (
  id SERIAL PRIMARY KEY,
  seller_id INTEGER NOT NULL REFERENCES users(id),
  title VARCHAR(255) NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  make VARCHAR(100) NOT NULL DEFAULT '',
  model VARCHAR(100) NOT NULL DEFAULT '',
  year SMALLINT NOT NULL,
  starting_price NUMERIC(12,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'draft'
  CHECK (status IN ('draft', 'listed', 'sold')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS auctions (
  id SERIAL PRIMARY KEY,
  vehicle_id INTEGER NOT NULL REFERENCES vehicles(id) ON DELETE CASCADE,
  title VARCHAR(255) NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  status VARCHAR(20) NOT NULL DEFAULT 'draft'
  CHECK (status IN ('draft', 'active', 'ended')),
  starts_at TIMESTAMPTZ,
```

```

ends_at    TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS bids (
  id          SERIAL PRIMARY KEY,
  vehicle_id  INTEGER NOT NULL REFERENCES vehicles(id) ON DELETE CASCADE,
  user_id     INTEGER NOT NULL REFERENCES users(id),
  amount      NUMERIC(12,2) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

4.4 Migrations 002–008 — Schema Evolution

Run each migration in sequence. Each file solves a specific problem identified during development:

File	Change	Why
002	Add auction_id to bids, drop vehicle_id	Bids belong to auctions, not directly to vehicles
003	Add winning_bid_id FK to auctions	Records admin's winner selection
004	Add chassis_number, mileage, grade, images[] to vehicles	Required vehicle spec fields
005	Replace is_verified boolean with verification_status enum	Boolean can't represent pending state
006	Add min_increment to auctions	Enforce minimum bid raise per FR-11
007	Add reserve_price to auctions	Hidden minimum acceptable price
008	Add UNIQUE constraint on chassis_number	Prevent duplicate vehicle registrations

4.5 Apply All Migrations

```

psql -U postgres -d vehicle_auction -f Backend/migrations/001_initial_schema.sql
psql -U postgres -d vehicle_auction -f Backend/migrations/002_bids_auction_id.sql
psql -U postgres -d vehicle_auction -f Backend/migrations/003_auction_winner.sql
psql -U postgres -d vehicle_auction -f Backend/migrations/004_vehicle_fields.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/005_user_verification_status.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/006_auction_min_increment.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/007_auction_reserve_price.sql
psql -U postgres -d vehicle_auction -f
Backend/migrations/008_vehicle_chassis_unique.sql

```

4.6 Final Schema Reference

After all migrations, the final schema looks like this:

Table	Key Columns	Important Constraints
users	id, email, password_hash, name, role, verification_status, created_at	email UNIQUE; role IN ('admin','buyer'); verification_status IN ('pending','verified','rejected')
vehicles	id, seller_id, title, description, make, model, year, starting_price, status, chassis_number, mileage, grade, images[]	chassis_number UNIQUE (NULLs exempt); status IN ('draft','listed','sold'); images is TEXT[]
auctions	id, vehicle_id, title, status, starts_at, ends_at, min_increment, reserve_price, winning_bid_id	vehicle_id CASCADE DELETE; status IN ('draft','active','ended'); min_increment >= 0
bids	id, auction_id, user_id, amount, created_at	auction_id CASCADE DELETE; amount is NUMERIC(12,2)

4.7 Seed the Admin User

File: Backend/scripts/seed-admins.js — this creates the first admin account. Admin accounts cannot be created through the API.

```
import bcrypt from 'bcrypt';
import pg from 'pg';
import dotenv from 'dotenv';
dotenv.config();

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });
const hash = await bcrypt.hash('ChangeMe1', 10);
await pool.query(
  `INSERT INTO users (email, password_hash, name, role, verification_status)
  VALUES ($1, $2, $3, 'admin', 'verified')`,
  ['owner@example.com', hash, 'System Admin']
);
console.log('Admin created: owner@example.com / ChangeMe1');
await pool.end();
cd Backend && node scripts/seed-admins.js
```

⚠ Change the admin password immediately after first login in production. The seed script is only for development bootstrap.

5. Backend — Layered Architecture

The backend follows a strict four-layer pattern. Understanding this is critical before writing any code.

Layer	File Location	Responsibility
Routes	src/routes/	URL patterns + middleware chain. Zero business logic.
Controllers	src/controllers/	Call ONE service method. Send response or call next(err). Nothing else.
Services	src/services/	ALL business logic, authorization checks, cross-entity rules.
Repositories	src/repositories/	SQL queries only. No business logic whatsoever.

5.1 Configuration & Database Pool

File: src/config/index.js — centralizes all environment variable reading. Never call process.env directly from application code; always go through this module.

```
import dotenv from 'dotenv';
dotenv.config();

export default {
  port: process.env.PORT || 3000,
  databaseUrl: process.env.DATABASE_URL,
  jwtSecret: process.env.JWT_SECRET,
  jwtExpiresIn: process.env.JWT_EXPIRES_IN || '1h',
  bcryptRounds: Number(process.env.BCRYPT_ROUNDS || 10),
  env: process.env.NODE_ENV || 'development'
};
```

File: src/db/pool.js — a single shared PostgreSQL connection pool. The on('error') listener kills the process if a background connection dies unexpectedly.

5.2 Express Application Entry

File: src/app.js — creates the Express app, mounts middleware, mounts routes under /api, and adds error handlers.

File: src/server.js — imports app, calls listen, and starts the cron scheduler.

 *Keep app.js and server.js separate. This allows you to import app in integration tests without starting the HTTP server.*

5.3 Authentication Middleware

File: src/middleware/auth.js — two exported functions:

- **authRequired** — reads Authorization: Bearer <token> header. Calls jwt.verify(). On success, sets req.user to the decoded payload. On failure, returns 401.
- **requireRole(roles=[])** — must come after authRequired. Checks req.user.role is in the allowed roles array. Returns 403 if not.

The JWT payload contains: { sub: userId, role, email, isVerified }. The sub field (subject) is the user's database ID.

5.4 Validation Middleware

File: src/middleware/validate.js — wraps Joi schema validation into Express middleware:

- **validate(schema)** — validates req.body. Returns 400 with details array on failure. Strips unknown fields from the body (prevents pollution).
- **validateQuery(schema)** — same but for req.query, with convert: true to cast strings to numbers.

Define Joi schemas in the route files, not in controllers or services. Validation is a transport concern, not a business concern.

5.5 Repositories

Repositories contain only SQL. They receive plain values, run a parameterized query, and return the result. Rules:

- Never throw custom business errors from a repository — let the raw database error propagate up to the service layer
- Use parameterized queries (\$1, \$2, ...) — never string-concatenate user input into SQL
- Use RETURNING * on INSERT and UPDATE statements to return the full updated row
- Use pool.query() for single statements. For transactions, use a client from pool.connect()

5.6 Key Repository Patterns

User Repository

Functions: findByEmail, findById, create, findAll, setStatus. These are straightforward single-table queries.

Auction Repository

The findAll and findById queries join vehicles to include vehicle_make, vehicle_model, vehicle_year, and vehicle_images. They also include a correlated subquery for highest_bid:

```
(SELECT MAX(amount) FROM bids WHERE auction_id = a.id) AS highest_bid
```

This avoids a denormalized column on the auctions table that could fall out of sync with the bids table.

Vehicle Repository

The findAll function supports dynamic filtering with up to 9 filter conditions. The pattern is to build a conditions array and params array, then compose the WHERE clause. The query runs twice: once with COUNT(*) for pagination metadata, once for the actual data with LIMIT/OFFSET.

5.7 Services

Services contain all business logic. They are the most important layer to get right.

Auth Service

`register()` — validates role is buyer (admin cannot self-register), checks email uniqueness, hashes password with `bcrypt`, creates user.

`login()` — finds user by email, compares password with `bcrypt.compare()`, signs and returns JWT. The token includes `isVerified` so the frontend can show/hide the bid form without an extra API call.

Auction Service — On-Read Status Correction

This is a critical design decision. The cron scheduler updates auction statuses every minute, but it could miss a tick (server restart, etc.). The `effectiveStatus()` function re-checks timestamps on every read:

```
export function effectiveStatus(auction) {
  const now = new Date();
  if (auction.status === 'draft' && auction.starts_at && new
Date(auction.starts_at) <= now)
    return 'active';
  if (auction.status === 'active' && auction.ends_at && new Date(auction.ends_at)
<= now)
    return 'ended';
  return auction.status;
}
```

If the computed status differs from the stored status, the service immediately updates the database row. This dual approach (cron + on-read) ensures the displayed status is always accurate.

Auction Service — Reserve Price Privacy

`reserve_price` is in the database. The `stripReserve()` function removes it from the API response for non-admins:

```
function stripReserve(auction, user) {
  const reserveMet = auction.reserve_price == null ? null
    : (auction.highest_bid != null
      ? Number(auction.highest_bid) >= Number(auction.reserve_price)
      : false);
  if (user?.role === 'admin') return { ...auction, reserve_met: reserveMet };
  const { reserve_price, ...rest } = auction; // strip the actual amount
  return { ...rest, reserve_met: reserveMet }; // but keep the boolean indicator
}
```

Buyers can see whether the reserve has been met (shown as a colored badge in the UI) but never see the actual reserve price amount.

Bid Service

`placeBid()` performs these checks in order (each throws a descriptive error if it fails):

5. User is verified (`isVerified` from JWT payload)
6. Auction exists
7. `effectiveStatus(auction) === 'active'` (checks draft and ended cases separately for better error messages)
8. Bid amount $\geq$ highest bid + `min_increment` (or just $>$ highest if `min_increment` is 0)

5.8 Error Convention

Services throw Error objects with a `.status` property. The global error handler reads this property to set the HTTP status code. Pattern:

```
const err = new Error('Resource not found');
err.status = 404;
throw err;
```

Controllers are thin: call service, send result, or call `next(err)`. They contain no business logic:

```
export async function getAuctionController(req, res, next) {
  try {
    const item = await auctionService.getAuctionById(req.params.id, req.user);
    if (!item) return res.status(404).json({ message: 'Auction not found' });
    res.json(item);
  } catch (err) { next(err); }
}
```

5.9 The Scheduler

File: `src/scheduler.js` — a node-cron job that runs every minute (`* * * * *`). It fires two UPDATE queries:

```
// draft → active when starts_at has passed
UPDATE auctions SET status = 'active'
WHERE status = 'draft' AND starts_at IS NOT NULL AND starts_at <= NOW()

// active → ended when ends_at has passed
UPDATE auctions SET status = 'ended'
WHERE status = 'active' AND ends_at IS NOT NULL AND ends_at <= NOW()
```

 The scheduler is started in `server.js` after the HTTP server begins listening. It is NOT started in `app.js`, so test files importing `app.js` won't trigger it.

5.10 API Route Map

Method	Path	Auth	Description
POST	<code>/api/auth/register</code>	None	Register new buyer account
POST	<code>/api/auth/login</code>	None	Login → returns { token, user }
GET	<code>/api/users</code>	Admin	List all users with verification status
GET	<code>/api/users/me</code>	Auth	Get own profile from database
PATCH	<code>/api/users/:id/status</code>	Admin	Set verified or rejected
GET	<code>/api/vehicles</code>	None	Paginated, filterable vehicle list
GET	<code>/api/vehicles/:id</code>	None	Single vehicle detail
POST	<code>/api/vehicles</code>	Admin	Create vehicle listing
PUT	<code>/api/vehicles/:id</code>	Admin	Update vehicle listing
DELETE	<code>/api/vehicles/:id</code>	Admin	Delete vehicle
GET	<code>/api/auctions</code>	None	All auctions with computed status
GET	<code>/api/auctions/won/me</code>	Auth	Buyer's won auctions (must be BEFORE <code>/:id</code>)
GET	<code>/api/auctions/:id</code>	None	Single auction detail
POST	<code>/api/auctions</code>	Admin	Create auction

Method	Path	Auth	Description
PUT	/api/auctions/:id	Admin	Update auction
DELETE	/api/auctions/:id	Admin	Delete auction
POST	/api/auctions/:id/close	Admin	Close and auto-assign winner
POST	/api/auctions/:id/winner	Admin	Manually assign specific bid as winner
GET	/api/auctions/:id/winner	None	Get winner details
GET	/api/auctions/:id/bids	None	Bid history (public)
POST	/api/auctions/:id/bids	Verified Buyer	Place a bid

⚠ Route `/api/auctions/won/me` **MUST** be declared **BEFORE** `/api/auctions/:id` in the router file. Express matches routes top-to-bottom and 'won' would be treated as an ID parameter otherwise.

6. Frontend Architecture

The frontend is a React 19 single-page application using React Router v7 for client-side routing. It communicates with the backend exclusively via Axios.

6.1 Vite Configuration

File: Frontend/vite.config.js — configures the dev server proxy so /api requests from the browser are forwarded to the backend on port 3000. This avoids CORS issues during development:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: {
      '/api': { target: 'http://localhost:3000', changeOrigin: true }
    }
  }
})
```

6.2 Tailwind CSS v4 Setup

Tailwind v4 has NO tailwind.config.js. Configuration is done entirely in CSS. Create src/index.css:

```
@import "tailwindcss";
@plugin "@tailwindcss/forms";

@layer components {
  .input {
    @apply w-full border border-gray-300 rounded-lg px-3 py-2 text-sm
      focus:outline-none focus:ring-2 focus:ring-blue-500 bg-white;
  }
}
```

 *The .input utility class is reused across all forms. Define reusable component classes here rather than repeating long class strings everywhere.*

6.3 Application Entry — main.jsx

Wraps the app in StrictMode, BrowserRouter, and AuthProvider in that order. BrowserRouter must wrap AuthProvider so the auth context can access the router:

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>
)
```

6.4 Auth Context — The Most Important Frontend File

File: `src/context/AuthContext.jsx` — manages authentication state for the entire application. Key behaviours:

- Stores JWT in `localStorage` for persistence across page refreshes
- Decodes the JWT client-side using `atob()` — NO API call on page load. This is fast but means role/verification changes won't show until next login. The profile page calls `/users/me` to get current data.
- Auto-logout after 30 minutes of inactivity — a timer is reset on any user interaction event (mousemove, keydown, click, scroll, touch). On timeout, it dispatches a session-expired custom event and clears state.
- Exposes: `token`, `user` (decoded JWT payload), `login(token)`, `logout()`, `isAdmin`, `isVerified`

6.5 Routing

File: `src/router/index.jsx` — defines all routes. Public routes render directly. Protected routes are wrapped in `ProtectedRoute`:

Path	Component	Access
<code>/</code>	Redirect to <code>/auctions</code>	Public
<code>/login</code>	LoginPage	Public
<code>/register</code>	RegisterPage	Public
<code>/auctions</code>	AuctionListPage	Public
<code>/auctions/:id</code>	AuctionDetailPage	Public
<code>/auctions/:id/winner</code>	WinnerPage	Public
<code>/vehicles</code>	VehicleListPage	Public
<code>/vehicles/:id</code>	VehicleDetailPage	Public
<code>/admin</code>	AdminPage	Admin only
<code>/admin/vehicles/new</code>	VehicleFormPage	Admin only
<code>/admin/vehicles/:id/edit</code>	VehicleFormPage	Admin only
<code>/admin/auctions/new</code>	AuctionFormPage	Admin only
<code>/admin/auctions/:id/edit</code>	AuctionFormPage	Admin only
<code>/profile</code>	ProfilePage	Any authenticated user

6.6 API Layer

File: `src/api/client.js` — a single Axios instance with a request interceptor that reads the JWT from `localStorage` and attaches it as a Bearer token:

```
import axios from 'axios'
const client = axios.create({ baseURL: '/api' })
client.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

```
})
export default client
```

One API file per resource: auth.js, users.js, vehicles.js, auctions.js, bids.js, cloudinary.js. Each exports thin wrapper functions over the Axios client.

6.7 Cloudinary Image Upload

File: src/api/cloudinary.js — unsigned upload directly from the browser. No server involvement. Returns the secure_url of the uploaded image:

```
export async function uploadImage(file) {
  const body = new FormData()
  body.append('file', file)
  body.append('upload_preset', import.meta.env.VITE_CLOUDINARY_UPLOAD_PRESET)
  const res = await fetch(
    `https://api.cloudinary.com/v1_1/${import.meta.env.VITE_CLOUDINARY_CLOUD_NAME}/image/upload`,
    { method: 'POST', body }
  )
  if (!res.ok) throw new Error('Image upload failed')
  return (await res.json()).secure_url
}
```

6.8 Key Components

ProtectedRoute.jsx

Reads user from AuthContext. If no user, redirects to /login. If a role prop is specified and user.role doesn't match, redirects to /auctions. Otherwise renders children.

CountdownTimer.jsx

Takes an endsAt prop. Uses setInterval every 1000ms to recalculate time remaining. Returns null when time is up (shows Ended text). Always clear the interval in the useEffect cleanup function to prevent memory leaks.

StatusBadge.jsx

Maps status strings to Tailwind color classes. Handles: draft, active, ended, listed, sold, pending, verified, rejected.

Navbar.jsx

For admin users, fetches the count of pending users on mount and shows a red badge. Listens for a user-status-changed custom event (dispatched by AdminPage) to refresh the count when an admin approves/rejects without needing a page reload.

6.9 Page Descriptions

LoginPage.jsx

Reads location.state?.message for the session-expired redirect message and ?registered=1 query param for registration success. On successful login, decodes the JWT role and redirects: admin → /admin, buyer → /auctions.

AuctionDetailPage.jsx — Most Complex Page

Features:

- Lightbox image gallery: click hero image to open fullscreen. Arrow keys navigate. Escape closes. Click to zoom 150%.
- Polling: re-fetches auction data + bids every 15 seconds while status is active
- Reserve badge: shows colored card (blue=no reserve, red=not met, green=met). Admin sees actual LKR amount.
- Bid form: only shown for authenticated, non-admin, verified buyers while auction is active
- Admin controls: Close Auction button, winner selection table

VehicleListPage.jsx

URL-synced filters using useSearchParams. All filter state lives in the URL so bookmarking and sharing links works. Skeleton loading with pulsing grey cards. Smart pagination with ellipsis for large page counts.

AdminPage.jsx

Three tabs: Vehicles, Auctions, Users. The Users tab has Verify/Reject buttons per user. On action success, dispatches `window.dispatchEvent(new CustomEvent('user-status-changed'))` so the Navbar badge updates immediately.

App.jsx — Won Auctions Banner

On mount (when user changes), if the user is a buyer, fetches `/api/auctions/won/me`. Filters out auction IDs stored in `localStorage('wonDismissed')`. Displays a green banner for each un-dismissed win. Clicking the X adds the ID to `wonDismissed`.

7. Cloudinary Setup

Images are uploaded directly from the browser to Cloudinary. The database stores the returned CDN URL. No images are stored on your server.

7.1 Create a Free Account

9. Sign up at cloudinary.com
10. Go to Settings → Upload → Upload presets → Add upload preset
11. Set Signing Mode to Unsigned
12. Name it `vehicle_img` (must match `VITE_CLOUDINARY_UPLOAD_PRESET` in `.env`)
13. Copy your Cloud Name from the dashboard — use it for `VITE_CLOUDINARY_CLOUD_NAME`

7.2 How Upload Works in the UI

In `VehicleFormPage`, the file input calls `uploadImage()` from `cloudinary.js` on each selected file. The returned URL is added to a local `images` state array. On form submit, the full array of URLs is sent to the backend and stored in the `vehicles.images TEXT[]` column.

For editing, existing Cloudinary URLs are displayed with a hover-to-remove overlay. Newly selected files are previewed locally (using `URL.createObjectURL()`) until the form is submitted, at which point they are uploaded.

 *Never store image files on your backend server. Always use a CDN like Cloudinary. Local files would be lost on every deployment.*

8. Recommended Build Order

Build the system in this order. Each step produces something testable before you move on. Do not skip steps.

Step	What to Build	How to Test
1	Database: run all 8 migrations, run seed script	psql -d vehicle_auction -c '\dt' — should list 4 tables
2	Backend: config, db/pool, app.js, server.js, health route	npm run dev → GET /api/health returns 200
3	Backend: auth middleware, validate middleware, error handlers	Unit test by temporarily adding a protected route
4	Backend: userRepository, authService, authController, auth routes	POST /api/auth/register and POST /api/auth/login in Postman/curl
5	Backend: vehicleRepository, vehicleService, vehicleController, vehicle routes	CRUD operations in Postman — test with and without admin token
6	Backend: auctionRepository, auctionService, auctionController, auction routes	Create auction, check status transitions, test /won/me route ordering
7	Backend: bidRepository, bidService, bidController, bid routes (nested under auctions)	Place bid as verified buyer, test increment validation, test unverified rejection
8	Backend: scheduler	Create auction with starts_at 2 minutes in future, wait, check status changes to active
9	Frontend: Vite config, index.css, main.jsx, AuthContext, client.js	npm run dev — blank page with no console errors
10	Frontend: LoginPage, RegisterPage, router, ProtectedRoute	Register, login, verify redirect works by role
11	Frontend: AuctionListPage, AuctionDetailPage, CountdownTimer, StatusBadge	Browse auctions, see countdown timer on active auctions
12	Frontend: VehicleListPage, VehicleDetailPage	Filter vehicles, verify pagination URL-sync
13	Frontend: Bid form in AuctionDetailPage, 15s polling	Place a bid, see it appear in bid list, verify form hides for admin
14	Frontend: AdminPage tabs (Vehicles, Auctions, Users)	Verify user, check badge updates, close auction
15	Frontend: VehicleFormPage with Cloudinary upload	Create vehicle with 2+ images, verify URLs in DB
16	Frontend: AuctionFormPage	Create and edit auction, verify reserve price hidden from buyer
17	Frontend: ProfilePage, WinnerPage, App.jsx won banner	Full end-to-end: buy wins auction, see banner, see profile section

Step	What to Build	How to Test
18	Frontend: Navbar (pending count badge)	Approve a user, badge count decreases immediately

9. Common Mistakes and How to Avoid Them

9.1 Backend Mistakes

Forgetting ES Module Syntax

The backend uses type: "module" in package.json. This means all imports must use import/export syntax — not require(). All import paths must include the .js extension:

```
// X Wrong
const express = require('express')
import config from '../config'

// ✓ Correct
import express from 'express'
import config from '../config/index.js'
```

Business Logic in the Wrong Layer

The most common architectural mistake. If you find yourself writing an if statement in a controller or a SQL query in a service, stop and move it to the correct layer. Controllers call services. Services call repositories. Never skip a layer.

Not Parameterizing SQL

```
// X Never do this — SQL injection vulnerability
pool.query(`SELECT * FROM users WHERE email = '${email}'`)

// ✓ Always use parameterized queries
pool.query('SELECT * FROM users WHERE email = $1', [email])
```

Route Order in auctions.js

The /won/me route MUST come before the /:id route. Express matches top-to-bottom. If /:id comes first, won is treated as an auction ID:

```
// ✓ Correct order
router.get('/won/me', authRequired, getMyWonAuctionsController);
router.get('/:id', getAuctionController);
```

Not Setting type: module in package.json

Without this, Node.js treats .js files as CommonJS and import statements fail. Add "type": "module" to Backend/package.json.

9.2 Frontend Mistakes

Not Cleaning Up Intervals

CountdownTimer uses setInterval. Always return a cleanup function from useEffect:

```
useEffect(() => {
  const id = setInterval(() => setLeft(calcLeft(endsAt)), 1000)
  return () => clearInterval(id) // ← this prevents memory leaks
}, [endsAt])
```


Polling Without Cleanup

AuctionDetailPage polls every 15 seconds. The same cleanup rule applies. Also, only poll when status is active — stop polling for ended or draft auctions.

JWT Decoding vs. Fresh Data

AuthContext decodes the JWT stored in localStorage. This data is from when the user last logged in. If an admin verifies a buyer after they've already logged in, the buyer's isVerified in the token is still false. For the profile page, always call /api/users/me to get current data from the database, not just the token.

Forgetting to Dispatch Custom Events

AdminPage and Navbar communicate via a custom browser event, not React props or context. When you approve/reject a user in AdminPage, you must dispatch the event for the Navbar badge to update:

```
// In AdminPage after successful setStatus API call:  
window.dispatchEvent(new CustomEvent('user-status-changed'))
```

9.3 Database Mistakes

Editing Applied Migrations

Never edit a migration file after it has been run against any database. If you need to change the schema, create a new migration file (009_whatever.sql). Editing applied migrations breaks the ability to reproduce the schema from scratch.

Storing Images in the Database

The images column stores Cloudinary URLs (strings), not binary image data. Never store image binaries in PostgreSQL for a web application — it will destroy performance.

NULL Chassis Numbers and UNIQUE Constraint

PostgreSQL UNIQUE constraints treat each NULL as distinct — multiple rows can have NULL chassis_number. This is intentional (not all vehicles have chassis numbers yet). The service layer catches error code 23505 with constraint name vehicles_chassis_number_unique to return a clean 409 response when a duplicate non-null value is inserted.

10. Testing Your Implementation

10.1 Backend API Testing with curl

Use these curl commands to verify each endpoint. Replace <TOKEN> with the JWT from a login response:

Auth

```
# Register
curl -X POST http://localhost:3000/api/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"email":"buyer@test.com","password":"Password1","name":"Test Buyer","role":"buyer"}'

# Login
curl -X POST http://localhost:3000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"owner@example.com","password":"ChangeMe1"}'
# → Copy the token value for use below
```

Verify a User (Admin)

```
curl -X PATCH http://localhost:3000/api/users/2/status \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <ADMIN_TOKEN>' \
  -d '{"status":"verified"}'
```

Create Vehicle and Auction (Admin)

```
# Create vehicle
curl -X POST http://localhost:3000/api/vehicles \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <ADMIN_TOKEN>' \
  -d '{"title":"2020 Toyota Aqua","description":"Clean","startingPrice":1500000,"make":"Toyota","model":"Aqua","year":2020,"status":"listed","images":[]}'

# Create auction (replace vehicleId with ID from above)
curl -X POST http://localhost:3000/api/auctions \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <ADMIN_TOKEN>' \
  -d '{"vehicleId":1,"status":"active","endsAt":"2099-12-31T23:59:59Z","minIncrement":50000}'
```

Place a Bid (Verified Buyer)

```
curl -X POST http://localhost:3000/api/auctions/1/bids \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <BUYER_TOKEN>' \
  -d '{"amount":1600000}'
```

10.2 Frontend Smoke Test Checklist

Run through this checklist after completing the build:

14. Register a new buyer account — verify redirect to login with success message
15. Log in as admin — verify redirect to /admin

16. Create a vehicle with 2 images via Cloudinary upload
17. Create an active auction linked to that vehicle with a reserve price
18. Log in as buyer — verify bid form is hidden (unverified)
19. Verify buyer account as admin — dispatch user-status-changed event, verify Navbar badge decrements
20. Log in as buyer — verify bid form is now visible (re-login required for token refresh)
21. Place a bid — verify it appears in bid list, verify reserve badge updates
22. Close the auction as admin — verify status changes to ended, highest bid selected as winner
23. Navigate to /auctions/:id/winner — verify winner details are public
24. Log in as winning buyer — verify You Won! banner appears
25. Navigate to /profile — verify won auction appears with winning bid amount
26. Dismiss the banner — verify it doesn't reappear on next load

11. Deployment Guide

This guide covers deployment to a single Ubuntu 22.04 server. For production, you will also want to configure SSL (HTTPS) using Let's Encrypt.

11.1 Server Prerequisites

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Install PostgreSQL 16
sudo apt install -y postgresql postgresql-contrib

# Install Nginx
sudo apt install -y nginx

# Install PM2 (process manager for Node.js)
sudo npm install -g pm2
```

11.2 Database Setup on Server

```
sudo -u postgres psql
CREATE DATABASE vehicle_auction;
CREATE USER auction_user WITH ENCRYPTED PASSWORD 'strong_password_here';
GRANT ALL PRIVILEGES ON DATABASE vehicle_auction TO auction_user;
\q

# Upload and run migrations
psql -U auction_user -d vehicle_auction -f migrations/001_initial_schema.sql
# ... run all 8 migrations
```

11.3 Backend Deployment

```
# Clone or upload your project to /var/www/vehicle-auction
cd /var/www/vehicle-auction/Backend
npm install --production

# Create production .env
nano .env
# Set NODE_ENV=production, strong JWT_SECRET, production DATABASE_URL

# Run seed script
node scripts/seed-admins.js

# Start with PM2
pm2 start src/server.js --name vehicle-auction-api
pm2 save
pm2 startup # follow the printed command to enable on boot
```

11.4 Frontend Build

```
cd /var/www/vehicle-auction/Frontend
```

```
npm install
npm run build
# dist/ directory contains the static files to serve
```

11.5 Nginx Configuration

Create `/etc/nginx/sites-available/vehicle-auction`:

```
server {
    listen 80;
    server_name yourdomain.com;

    root /var/www/vehicle-auction/Frontend/dist;
    index index.html;

    # API requests → Node.js backend
    location /api/ {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Frontend SPA — all non-API routes serve index.html
    location / {
        try_files $uri $uri/ /index.html;
    }
}
sudo ln -s /etc/nginx/sites-available/vehicle-auction /etc/nginx/sites-enabled/
sudo nginx -t # test config
sudo systemctl reload nginx
```

💡 The `try_files /index.html` fallback is critical for React Router. Without it, refreshing a page like `/auctions/5` returns a 404 from Nginx.

11.6 Production Checklist

27. `NODE_ENV=production` is set in backend `.env`
28. `JWT_SECRET` is a randomly generated 64-character string (not the development value)
29. Admin password changed from `ChangeMe1` to something strong
30. Database user has only `GRANT ALL` on `vehicle_auction` — not superuser
31. Firewall allows only ports 22 (SSH), 80 (HTTP), 443 (HTTPS)
32. PM2 is configured to restart the API on crash and on server reboot
33. Nginx serves the React `dist/` folder as static files
34. SSL/HTTPS configured via Let's Encrypt certbot

12. Quick Reference

12.1 File Naming & Location Reference

File	Purpose
Backend/src/config/index.js	All environment variables — the only place process.env is read
Backend/src/db/pool.js	Shared PostgreSQL connection pool instance
Backend/src/app.js	Express app setup — middleware, routes, error handlers
Backend/src/server.js	HTTP server start + scheduler initialization
Backend/src/scheduler.js	Cron job for automatic auction status transitions
Backend/src/middleware/auth.js	authRequired and requireRole middleware
Backend/src/middleware/validate.js	Joi schema validation middleware
Backend/src/middleware/errorHandler.js	Global error handler — reads err.status
Backend/src/repositories/*.js	SQL queries only — one file per database table
Backend/src/services/*.js	Business logic, authorization, cross-entity rules
Backend/src/controllers/*.js	Thin: call service, send response or next(err)
Backend/src/routes/*.js	URL patterns + middleware chains
Backend/scripts/seed-admins.js	One-time admin user creation
Backend/migrations/*.sql	Numbered schema changes — never edit after applying
Frontend/src/context/AuthContext.jsx	JWT storage, decode, auto-logout — the auth brain
Frontend/src/api/client.js	Axios instance with Bearer token interceptor
Frontend/src/api/cloudinary.js	Unsigned image upload to Cloudinary CDN
Frontend/src/router/index.jsx	All route definitions and ProtectedRoute wrapping
Frontend/src/components/CountdownTimer.jsx	Live countdown with interval cleanup
Frontend/src/components/Navbar.jsx	Pending user badge + session-expired listener

12.2 Error Response Format

Every error from the API follows this format:

```
{ "message": "Human-readable description",
  "details": ["optional", "array", "only present for validation errors"] }
```

HTTP Status	Meaning	Example Cause
400	Bad input / business rule violation	Bid too low, auction not active, missing field
401	Missing or invalid token	No Authorization header, expired JWT
403	Wrong role or unverified	Buyer tries admin route, unverified buyer bids

HTTP Status	Meaning	Example Cause
404	Resource not found	Auction ID does not exist
409	Conflict	Duplicate email, winner already set, duplicate chassis
500	Unexpected server error	Database connection lost, unhandled exception

12.3 JWT Payload Structure

```
// What jwt.sign() encodes:
{ sub: 42,                // user.id (database primary key)
  role: 'buyer',          // 'admin' or 'buyer'
  email: 'u@test.com',
  isVerified: true,       // verification_status === 'verified'
  iat: 1234567890,        // issued at (added by jwt automatically)
  exp: 1234571490 }       // expiry (added by jwt automatically)
```

12.4 Key Design Decisions Summary

Decision	What Was Done	Why
Auction status	Cron every minute + on-read correction	Guarantees accuracy even if scheduler misses a tick
Reserve price	Stripped from API response for non-admins; reserve_met boolean always included	Buyers see whether reserve is met without knowing the amount
highest_bid	Computed at query time with correlated subquery	Avoids denormalized column that could go out of sync
Admin creation	Script only — role: admin blocked in register API	Prevents malicious self-registration as admin
Images	TEXT[] column of Cloudinary URLs	No binary data in DB; CDN handles delivery
Verification	verification_status enum (pending/verified/rejected)	Boolean can't represent the pending state
Won banners	Stored dismissed IDs in localStorage wonDismissed key	Persistent across browser sessions without a DB table
Cross-component events	CustomEvent on window	Avoids prop-drilling for Navbar badge ↔ AdminPage sync